

1 Program

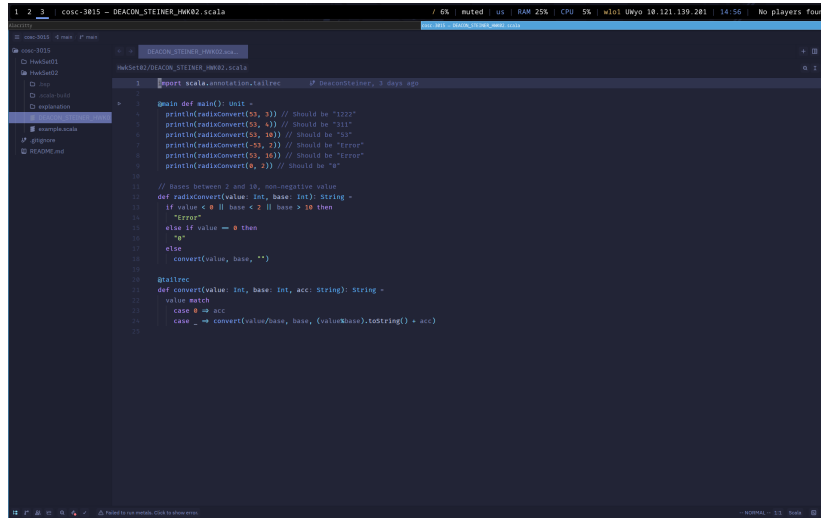


Figure 1: Assignment Solution

This program was relatively easy to implement. The only piece I struggled with was while learning about tail recursion and accumulators. This was a fairly new concept for me coming from more imperative languages.

The program itself is a fairly standard general solution for radix conversion. It repeatedly divides the target integer by the desired base, prepending the remainders to the acc string, which collects the digits from least to most significant, hence prepending.

I took advantage of Scala's strong pattern matching to match on the value to ensure the edge cases were handled properly

This is a very strong example of using functional principles, both `radixConvert` and the `convert` helper are pure, and `convert` even takes advantage of an accumulator to avoid mutability issues and ensures full tail recursion.

